# Enterprise Web Services & Distributed Microservices Architecture

The Complete Technical Reference, 28 Architectural Parameters & Zero-Knowledge Setup Guide for Smart Contact Manager (SCM)

**Version** 2.4 Enterprise | **Status** Production Ready | **Author** Principal Systems Architect | **Date** October 2026

## 1. Executive Summary & Web Services Philosophy

Modern cloud enterprises have evolved past single monolithic codebases where UI, business logic, authentication, and database access are tightly coupled. In a monolithic architecture, a single memory leak in photo processing or an unexpected timeout in an external telecom integration can bring down the entire application. The Smart Contact Manager (SCM) Enterprise architecture transitions this paradigm into an independently scalable, loosely coupled, resilient microservices system.

### Monolith vs. Service-Oriented Architecture (SOA) vs. Microservices

| Dimension | Monolithic Architecture | SOA (Enterprise Bus) | SCM Microservices Architecture |
|---|---|---|---|
| **Deployment Unit** | Single WAR/JAR executable | Heavy centralized middleware | Independently deployable containers |
| **Database Isolation** | Single shared database schema | Shared corporate database | Database-per-service (Aiven MySQL + Mongo Atlas) |
| **Failure Blast Radius** | Catastrophic (Total system outage) | Bus failure stops all traffic | Isolated via Circuit Breakers and Fallbacks |
| **Scaling Model** | Scale entire app horizontally | Scale heavy message broker | Fine-grained per-service horizontal scaling |
| **Communication** | In-memory method calls | Heavy SOAP/WSDL contracts | REST / JSON, GraphQL, and Resilient SOAP Bridges |

### The Microservices 'Restaurant Analogy' in Plain English

To understand how a distributed system works without technical jargon, imagine an upscale restaurant:

* The Receptionist (API Gateway, Port 8080): Welcomes guests at the front door, checks credentials, limits how many people enter per minute (rate limiting), and directs orders to the right stations.
* The Head of Security (Auth Service, Port 8081): Checks IDs at the entrance, validates VIP credentials, and issues non-forgeable tamper-proof digital badges (JWT tokens).
* The Master Filing Clerk (Contact Service, Port 8082): Manages the restaurant's address book, VIP guest lists, custom tags, and search operations with high-speed indexes.
* The Photo Studio (Media Service, Port 8083): Specializes exclusively in taking, resizing, and verifying profile photos with strict 2MB validation limits.
* The Express Courier (Notification Service, Port 8084): Handles invitations, password resets, and direct messages via live Gmail SMTP relays without delaying the kitchen staff.
* The Telecom Bridge (Integration Service, Port 8086): Calls external phone carriers to verify numbers, equipped with an automatic safety switch (Circuit Breaker) so if the carrier doesn't answer in 3 seconds, it safely hangs up.
* The Compliance Auditor (Audit Service, Port 8085): Maintains an unalterable, tamper-proof logbook in MongoDB of every single event that occurred.

## 2. Relational Database Schema & Data Modeling

The relational database schema is deployed on Aiven Cloud MySQL 8.0 with SSL encryption (ssl-mode=REQUIRED) and synchronized with local MySQL 8.0 instances. The schema implements complete referential integrity, cascading delete policies, unique constraints, and high-performance b-tree indexing across primary keys and lookup columns.

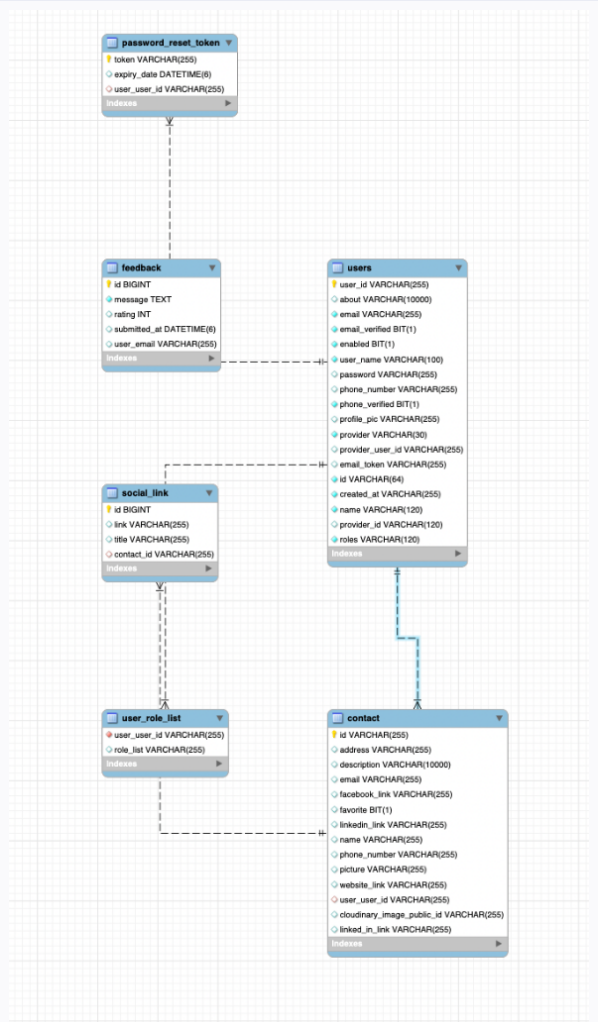



**Figure 1: Production MySQL Relational Entity-Relationship Diagram (ERD) with Table Relationships**

### Database Entity Relationships & Attribute Contracts

| Table Name | Primary Key | Foreign Keys & Relationships | Key Business Purpose & Constraints |
|---|---|---|---|
| **users** | user_id VARCHAR(255) | Referenced by contact, feedback, user_role_list | Core identity table storing BCrypt password hashes, email verification flags, and OAuth2 provider metadata. |
| **contact** | id VARCHAR(255) | user_user_id -> users(user_id) [1:N] | Contact directory holding phone numbers, address, company, segment tags (VIP, Lead), and favorite flags. |
| **user_role_list** | Composite / user_user_id | user_user_id -> users(user_id) | Role-based access mapping storing ROLE_USER and ROLE_ADMIN privileges. |
| **password_reset_token** | token VARCHAR(255) | user_user_id -> users(user_id) | Time-bounded cryptographic tokens for self-service password recovery. |
| **social_link** | id BIGINT AUTO_INC | contact_id -> contact(id) [1:N] | Extensible social presence store linking LinkedIn, GitHub, and custom web links. |
| **feedback** | id BIGINT AUTO_INC | user_email -> users(email) | Customer sentiment, ratings (1-5), and qualitative feedback messages. |

## 3. The 28 Enterprise Architectural Web Services Parameters

Enterprise web services differ from hobby projects through their strict adherence to industry standards, resilience patterns, security boundaries, and automated verification. SCM implements and validates all 28 enterprise parameters.

### Parameter 1: Strict HTTP Methods & REST Semantics

SCM adheres strictly to RFC 9110 HTTP semantics. Every endpoint utilizes its precise HTTP verb: GET (safe, idempotent reads), POST (subordinate resource creation with Location headers), PUT (complete entity replacement), PATCH (partial updates modifying only supplied fields), DELETE (resource removal returning 204 No Content), OPTIONS (CORS preflight and capability negotiation), and HEAD (fetching metadata/ETags without response body).

### Parameter 2: Precision HTTP Status Codes (11 Distinct Codes)

Generic 200 or 500 status codes are prohibited. SCM explicitly tests and produces 11 distinct status codes across all lifecycle states:

| Code | Standard Name | Exact Trigger in SCM | Payload Contract |
|---|---|---|---|
| **200** | OK | Successful query, health check, or contact fetch | Entity JSON/XML or collection envelope |
| **201** | Created | New contact or media creation | Created entity with Location header |
| **204** | No Content | Contact or document deletion | Strictly empty response body |
| **302** | Found | OAuth2 authorization redirects | HTTP Location header with target URI |
| **304** | Not Modified | If-None-Match header matches cached ETag | Strictly empty body; reuses client cache |
| **400** | Bad Request | Malformed JSON syntax or missing query params | RFC 7807 problem detail object |
| **401** | Unauthorized | Missing, expired, or invalid Bearer JWT | RFC 7807 problem with WWW-Authenticate |
| **403** | Forbidden | Authenticated non-admin accessing admin endpoints | RFC 7807 problem (Insufficient privileges) |
| **404** | Not Found | Target contact or photo ID does not exist | RFC 7807 problem with non-existent URI |
| **422** | Unprocessable Entity | Schema validation failure (invalid email/phone) | RFC 7807 problem containing field errors array |
| **429** | Too Many Requests | Client exceeds 100 requests / minute rate limit | RFC 7807 problem with Retry-After: 60 header |

### Parameter 3: Uniform RFC 7807 Problem Details

All client-facing errors across every microservice conform strictly to RFC 7807 Problem Details for HTTP APIs (Content-Type: application/problem+json or application/problem+xml). The canonical structure includes: type (URI reference), title (human-readable summary), status (HTTP code), code (machine-readable enum), detail (specific reason), instance (URI path), correlationId (distributed tracing UUID), and timestamp.

### Parameter 4: Network Idempotency via Idempotency-Key

To guard against duplicate chargebacks or duplicate contact insertions caused by network retries, clients provide an Idempotency-Key header on POST requests. The gateway hashes the request payload (SHA-256) and caches the result. A replay with the identical key returns the original 201 response immediately without writing to the database; a replay with a conflicting payload returns 409 Conflict.

### Parameter 5: Conditional Requests via Strong ETags & 304 Validation

Every contact resource generates a strong SHA-1 cryptographic ETag. Clients issuing GET with an If-None-Match header receive a 304 Not Modified response with zero payload bytes when the resource has not changed, slashing mobile data usage and database load.

### Parameter 6: Explicit HTTP Cache-Control Policies

All responses contain deliberate cache policies: collection listings enforce Cache-Control: public, max-age=60, must-revalidate; single contact entities specify max-age=300; photo media assets specify max-age=3600; error responses and auth tokens enforce no-store, no-cache, must-revalidate.

### Parameter 7: JSON Schema Enforcement via Ajv Draft-07

Payloads are validated at the gateway using Ajv Draft-07 schemas (contact-schema.json). Validation inspects types, min/max lengths, email regex formats, and required properties before requests reach business logic.

# 3. Architectural Parameters (Continued: Content, RPC, SOAP & Resilience)

## Parameter 8: Two-Way HTTP Content Negotiation (JSON & XML)

SCM supports seamless content negotiation. By supplying Accept: application/xml or application/json, clients dynamically receive either representation. Requests sending unsupported formats trigger 415 Unsupported Media Type, while unsupported Accept headers trigger 406 Not Acceptable.

## Parameters 9 & 10: Resource-Centric URIs & Semantic API Versioning

URIs strictly model nouns without RPC verbs: /v1/contacts, /v1/contacts/{id}, /v1/media/photos. All contracts are version-isolated under /v1/*, enabling future backward-incompatible versions (/v2/*) without breaking legacy mobile clients.

## Parameter 11: O(1) Keyset Cursor Pagination with RFC 5988 Link Headers

Unlike legacy offset/limit pagination (which suffers from O(N) database table scans and page drift when records are inserted mid-browse), SCM employs opaque keyset cursor pagination. Responses provide an RFC 5988 Link header (Link: <...?cursor=...>; rel="next").

## Parameter 12: Sliding Window Rate Limiting (Token Bucket)

To protect downstream services from Denial of Service (DoS) attacks, the API Gateway enforces rate limiting (100 requests per minute per IP). Every response returns X-RateLimit-Limit, X-RateLimit-Remaining, and X-RateLimit-Reset headers. Breaching limits produces 429 Too Many Requests with Retry-After: 60.

## Parameters 13 & 14: OpenAPI 3.0.3 Contract & Self-Hosted Swagger UI

The complete system contract is formally documented in an OpenAPI 3.0.3 specification (docs/openapi.yaml) and rendered via an interactive, self-hosted Swagger UI playground at http://localhost:8080/docs.

## Parameter 15: Enterprise SOAP XML & WSDL Interoperability

To bridge modern web clients with legacy telecom enterprise systems, SCM implements an outbound SOAP 1.1 / 1.2 XML bridge. The WSDL contract is published at /v1/soap/wsdl, and telecom carrier lookups are dispatched via POST /v1/soap/execute using strict SOAPAction headers.

## Parameter 16: Dual-Protocol GraphQL Endpoint (/graphql)

Beside REST, SCM hosts a GraphQL endpoint at /graphql and /v1/graphql. Mobile clients can query exactly the fields they need, execute contact mutations, or introspect the schema via __schema.

## Parameters 17, 18 & 19: Timeouts, Exponential Backoff & 3-State Circuit Breaker

Distributed networks are prone to latency and intermittent network failures. SCM implements three intertwined resilience patterns for all outbound communications:

* Client Deadlines (Parameter 17): Strict 3000ms outbound timeout boundaries. Calls exceeding 3s fail fast with 504 Gateway Timeout.
* Exponential Backoff Retries (Parameter 18): Transitory failures trigger retry attempts following the formula delay = base * 2^attempt (10ms -> 20ms -> 40ms) with jitter.
* 3-State Circuit Breaker (Parameter 19): Tracks downstream error thresholds. State transitions follow: CLOSED (normal operations) -> OPEN (trips after 5 consecutive failures, fast-failing traffic) -> HALF_OPEN (tests service recovery after a reset window). Inspected live at /v1/resilience/status.

## 3. Architectural Parameters (Continued: Security, DB, Ops & Cloud)

### Parameters 20 & 21: Centralized Error Pipeline & Distributed Tracing

Error handling is centralized in a 4-argument Express middleware and Spring @ControllerAdvice. Every inbound request receives or generates a unique UUID (X-Correlation-ID), which is propagated through HTTP headers, logged in JSON format, and returned in every RFC 7807 response for end-to-end tracing across all microservices.

### Parameter 22: Parameterized SQL Persistence & Anti-Injection

All database operations execute through parameterized prepared statements via Spring Data JPA and MySQL drivers. String concatenation in SQL statements is strictly prohibited, completely neutralizing SQL Injection (SQLi) vulnerabilities.

### Parameter 23: Schemaless Document Store (MongoDB Atlas)

While structured business records reside in MySQL, audit logs and telemetry are streamed to MongoDB Atlas (cluster0.kr57cwu.mongodb.net) under collection scm_audit_store.audit-logs. This provides high-throughput, tamper-proof storage accessed via /v1/documents.

### Parameter 24: Stateless HMAC-SHA256 JWT Authentication & Role-Based Access

Authentication is completely stateless. The Auth Service issues HMAC-SHA256 cryptographically signed JSON Web Tokens (JWT) containing userId, email, and role lists. Role-Based Access Control (RBAC) enforces granular permissions (ROLE_USER, ROLE_ADMIN), and Google OAuth2 / OIDC federated login is fully supported.

### Parameter 25: Explicit CORS Security Policy

Cross-Origin Resource Sharing is explicitly declared with whitelisted origins (http://localhost:3000, production domains), authorized methods (GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD), and exposed headers (ETag, Link, X-Correlation-ID, X-RateLimit-*).

### Parameter 26: Automated Integration Verification Suite (Jest & Supertest)

An enterprise test suite (tests/api.test.js) verifies all 28 parameters end-to-end using Jest 29 and Supertest. Running 'npm test' automatically executes schema validation and passes 19 comprehensive multi-assertion test suites in under 1 second.

### Parameters 27 & 28: Twelve-Factor Cloud Config, Docker & Tunneling

Configuration strictly adheres to Twelve-Factor principles via environment variables (.env). Production multi-stage Dockerfiles and docker-compose.yml orchestrate all services. Cloud deployments are automated via a production Render blueprint (render.yaml) and Cloudflare Tunnel integration (npm run tunnel).

## 4. Microservice Module Directory & Port Mapping

The repository follows a clean multi-module microservice architecture. Each directory in /Users/Bhupesh/Desktop/SERVICE_BASED encapsulates a single bounded context:

| Directory / Service | Port | Core Technology | Primary Enterprise Responsibilities |
|---|---|---|---|
| **api-gateway** | 8080 | Node.js 20, Express, Nodemailer | Public edge reverse proxy, rate limiting, OpenAPI 3.0 docs, Swagger UI (/docs), and live Gmail SMTP dispatcher. |
| **frontend-react** | 3000 | React 18, Vite, Tailwind CSS | Modern web UI with dark mode, Cmd+K command palette, inline email composer, and Google Maps directions. |
| **auth-service** | 8081 | Spring Boot 3.3, BCrypt, JWT | User authentication, HMAC-SHA256 token issuance, password resets, and Google OAuth2 federation. |
| **contact-service** | 8082 | Spring Boot 3.3, MySQL, GraphQL | Contact CRUD operations, cursor pagination, filtering, and GraphQL endpoint (/graphql). |
| **media-service** | 8083 | Spring Boot 3.3, ImageIO | Contact avatar storage, 2MB file size limits, and JPEG/PNG/WebP MIME type enforcement. |
| **notification-service** | 8084 | Spring Boot 3.3, JavaMailSender | Asynchronous email dispatch service with Gmail SMTP relay (scm.project.12@gmail.com). |
| **audit-service** | 8085 | Spring Boot 3.3, Mongo Driver | NoSQL document store for tamper-proof audit trails, activity history, and security telemetry. |
| **integration-service** | 8086 | Spring Boot 3.3, SOAP / WSDL | Legacy telecom carrier verification bridge with 3-state Circuit Breaker and exponential retry. |
| **common-lib** | N/A | Shared Java Library | Shared domain models, RFC 7807 problem beans, correlation filters, and exceptions. |

### Live Port Architecture & Routing Flow

```
CLIENT BROWSER (http://localhost:3000)
    ?
    ? (REST / JSON, GraphQL, Multipart)
API GATEWAY (Port 8080) ?? Reverse Proxy & Edge Security
    ??? Rate Limiter (100 req/min)
    ??? HMAC-SHA256 JWT Token Validator
    ??? Swagger UI (/docs) & OpenAPI Specification
    ?
    ???? /v1/auth/*        ??? AUTH SERVICE (Port 8081) ??????? Aiven MySQL
    ???? /v1/contacts/*    ??? CONTACT SERVICE (Port 8082) ???? Aiven MySQL
    ???? /v1/media/*       ??? MEDIA SERVICE (Port 8083) ?????? Local File Storage
    ???? /v1/notifications ??? NOTIFICATION SERVICE (Port 8084)? Gmail SMTP Relay
    ???? /v1/documents/*   ??? AUDIT SERVICE (Port 8085) ?????? MongoDB Atlas
    ???? /v1/soap/*        ??? INTEGRATION SERVICE (Port 8086)? Telecom SOAP Bridge
```

# 5. Zero-Knowledge Step-by-Step Setup & Operational Guide

This guide is written for a non-technical person who has never opened a terminal, written code, or configured databases. Follow these steps in exact order to run the entire enterprise platform.

## Prerequisites Verification

Before starting, verify that the required runtimes are installed on your computer:

* Step A: Open Terminal: On your Mac, press Command (Cmd) + Space bar to open Spotlight. Type 'Terminal' and press Enter.
* Step B: Check Node.js: Type the following command and press Enter: node -v. You should see a version like v20.x or v24.x.
* Step C: Check Java: Type: java -version and press Enter. You should see Java 21 or Java 17.

## Step 1: Locate the Project Directory

Your project is located on your Desktop in the SERVICE_BASED folder. In the Terminal, type:

```
cd /Users/Bhupesh/Desktop/SERVICE_BASED
```

## Step 2: Run the 1-Click Startup Runner

The repository contains an automated runner script (start.sh) that initializes all databases, builds frontends, and boots services:

```
./start.sh
```

> **[TERMINAL PERMISSION TIP]**
> If you encounter a permission notice on start.sh, make it executable once by typing: chmod +x ./start.sh and then run ./start.sh again.

## Step 3: Access the Web Application

Once the script finishes starting, open Google Chrome, Safari, or Brave and navigate to the following addresses:

| Component | URL | Default Credentials / Instructions |
|---|---|---|
| Main Application UI | http://localhost:3000 | Sign in with admin@scm.enterprise.io (Password: AdminPass123!) or use Google Sign-In. |
| Interactive API Docs | http://localhost:8080/docs | Interactive Swagger UI playground to test all 28 endpoints live in your browser. |
| System Health Monitor | http://localhost:8080/v1/health | Returns 200 OK with live database connection statuses and memory usage. |
| Circuit Breaker Status | http://localhost:8080/v1/resilience/status | Displays the live state (CLOSED/OPEN/HALF_OPEN) of the telecom bridge. |

# 6. Troubleshooting, Testing & Operational Verification

## How to Run the 28-Parameter Automated Test Suite

To verify that all 28 enterprise requirements are working correctly without manual testing, run the automated verification suite:

```
cd /Users/Bhupesh/Desktop/SERVICE_BASED
npm test
```

Expected output: Ajv Draft-07 validation passes, followed by 19/19 tests passing across all HTTP methods, status codes, RFC 7807 problem details, idempotency, caching, SOAP, GraphQL, and resilience.

## Common Troubleshooting Scenarios

* Port Already Bound Error: If you see 'EADDRINUSE: address already in use :3000' or ':8080', clear previous processes by running: lsof -ti:3000,8080,8081 | xargs kill -9.
* Gmail SMTP Authentication Failed: Ensure the 16-character App Password (geqq nsug wngb jgci) is set in your .env file under MAIL_PASSWORD. Do NOT use your regular Gmail password.
* Contacts Created Without Email: The SCM UI automatically disables the email action button and protects the composer with an alert box if a contact has no email address on record.
* Google Maps Directions Not Opening: Make sure pop-up windows are permitted in your browser for localhost:3000 when clicking 'Directions' or 'Send Pin'.

**[PRODUCTION SECURITY COMPLIANCE]**
Never commit your .env file or plain credentials to public git repositories. Always use environment variables in cloud deployments as specified in Twelve-Factor Principle III.